

Lab 8: K-means Clustering

DT8008 VT26, Halmstad University

Binay Kumar Sah, binsah25@student.hh.se, Master's in Embedded and Intelligence System
Anton Bauer, antibau25@student.hh.se, TAEIS 25

March 2026

Introduction

In this lab we explore the K-means clustering algorithm and see how it can be used for practical applications like image compression. In Part A, we tackle the core functions for the K-means algorithm from scratch and test it on a simple 2D dataset. In Part B, we take what we built and apply it to a real image by using the algorithm to group similar colors together and compress the image by reducing its color palette to only the 16, 5 and 2, most common colors.

Part A: Implementing K-means and 2D Dataset Clustering

Objective:

The objective for Part A was to understand the inner workings of K-means rather than just using a pre-built library. We had to write the functions that find the closest centroids for each data point and then compute the new centroid means. We then utilized it to visually cluster a 2D dataset.

Key Concepts:

1. **K-means Clustering:** An unsupervised learning method that groups data into a specified number (K) of clusters. It starts with random guesses for the cluster centers (centroids) and then iteratively refines them, as described in the following concepts: Cluster Assignment Step and Move Centroid Step.
2. **Cluster Assignment Step:** In this step the algorithm looks at every single training example and assigns it to the closest centroid. We did this by calculating the Euclidean distance between the data point and all available centroids, picking the one with the smallest distance.
3. **Move Centroid Step:** When all points are assigned to a cluster the centroid itself is to be updated; it is done by taking the average (mean) of all the data points that were just assigned to it, moving the centroid to the actual center of that cluster.

Methods/Algorithm:

Algorithm 1 K-means Clustering Implementation

- 1: **Step 1: Initialization**
 - 2: Initialize K cluster centroids (often by randomly picking K existing data points).
 - 3: **Step 2: Iterative Optimization**
 - 4: Loop over a set number of iterations (e.g., $max_iterations = 50$).
 - 5: **Step 3: Cluster Assignment**
 - 6: For each point, calculate the Euclidean distance to all K centroids and assign the point to the closest one.
 - 7: **Step 4: Recompute Centroids**
 - 8: Calculate the new position of each centroid by averaging the coordinates of all points assigned to it.
 - 9: **Step 5: Visualization**
 - 10: Run the algorithm on the provided 2D dataset ($K = 3$) and plot the points, coloring them based on their final cluster assignment to see if the algorithm worked.
-

Observation and Results:

Before doing anything complex, we tested our implementation on a tiny dataset with just 4 points and $K = 3$ centroids. This was to make sure our code successfully returned the expected closest centroid indices and calculated the correct updated means.

Then we ran our K-means algorithm on a larger 2D dataset containing 300 data points instead. We let $K = 3$ remain and let it run for a maximum of 50 iterations. When we plotted the results we could clearly see the data points grouped into three rather distinct clusters (distinguished by three different colors and markers), as illustrated in Figure 1.

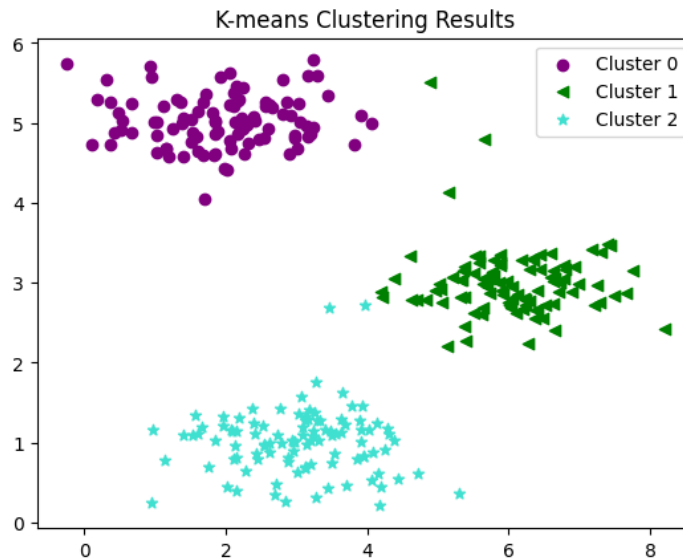


Figure 1: Visualization of the K-means algorithm applied to a 2D dataset. The data points look to be pretty successfully grouped into $K = 3$ distinct clusters (some outliers), each represented by a different color.

Part B: Image Compression with K-means

Objective:

The objective of Part B was more a fun and practical application: using our K-means code to compress a 24-bit RGB image down to just 16 colors. This allowed us to see how unsupervised learning can be used to drastically reduce file size while keeping the visual information mostly intact.

Key Concepts:

1. **RGB Encoding:** Images usually represents colors using three 8-bit values (0-255) for Red, Green, and Blue. This means there are ~ 16.7 millions of possible colors per pixel; as 24 bits per pixel $2^{24} \approx 16.7$ million, or as each color 0-255 resulting in $256 \times 256 \times 256 \approx 16.7$ million.
2. **Color Quantization:** Instead of grouping 2D coordinates like in Part A, we treated every single pixel's RGB values as a 3D data point (as three colors). K-means then finds the 16 average colors that best represent all the pixels in the image.
3. **Compression Factor:** By narrowing it down to a palette of 16 colors, we only need to store those 16 specific RGB values once. Then for each pixel we only need 4 bits to store the index (0-15) of its color, rather than the full 24 bits. This drops the image size by roughly 83%.

Methods/Algorithm:

Algorithm 2 Image Compression using K-means

- 1: **Step 1: Load and Reshape Image**
 - 2: Load the RGB image (a 3D matrix) and reshape it into a flat $n \times 3$ matrix of pixels. For our 128×128 image, this gave us 16384 individual 3D data points.
 - 3: **Step 2: Run K-means**
 - 4: Run the K-means function on the reshaped matrix to find the $K = 16$ best color centroids.
 - 5: **Step 3: Assign Pixels**
 - 6: Use our `findClosestCentroids` function to map every pixel in the original image to its closest new centroid color.
 - 7: **Step 4: Reconstruct Image**
 - 8: Replace the original pixel values with the RGB values of their assigned centroids and plot the resulting compressed image next to the original one.
-

Observation and Results:

We ran the compression on the provided image of a bird that was resized to 128×128 pixels. As this meant processing 16384 data points in 3D space, running the K-means algorithm to find the 16 clusters took a bit to execute. It was interesting to plot the final compressed image next to the original, as even though we threw away about 83% of the color data (compressing it from 24 bits per pixel to just 4 bits), the compressed image still looked much like the original bird, just slightly blockier/pixelated and less detailed (see Figure 2).

We also experimented with changing K to further reduce the file size (trying $K = 5$ and $K = 2$). As shown in Figure 3, using only 5 colors still preserves the most prominent visual structure. However, when we reduced the palette to just $K = 2$ colors (Figure 4), the compression got of course better, but the image started looking highly distorted and the reconstruction became mostly unintelligible.

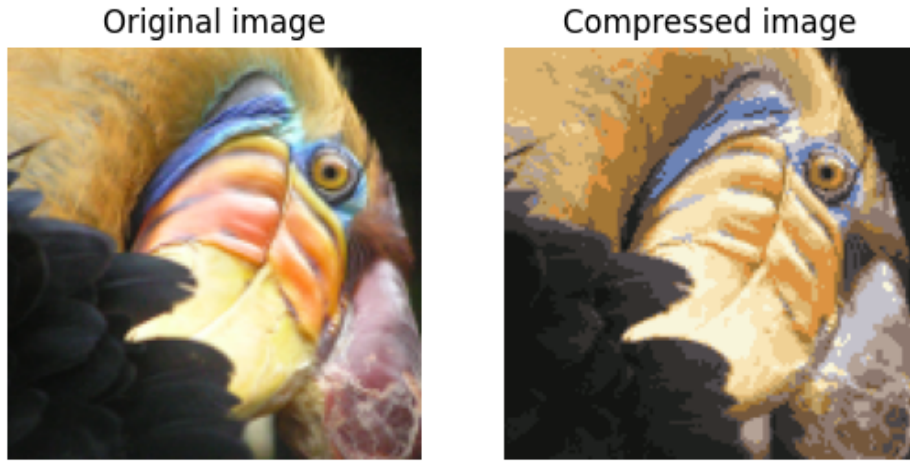


Figure 2: Comparison of the original 24-bit bird image (left) and the K-means compressed version using $K = 16$ colors (right). Reducing the color palette to 16 centroids (16 colors) achieves roughly an 83% reduction in size while maintaining the main visual structure.

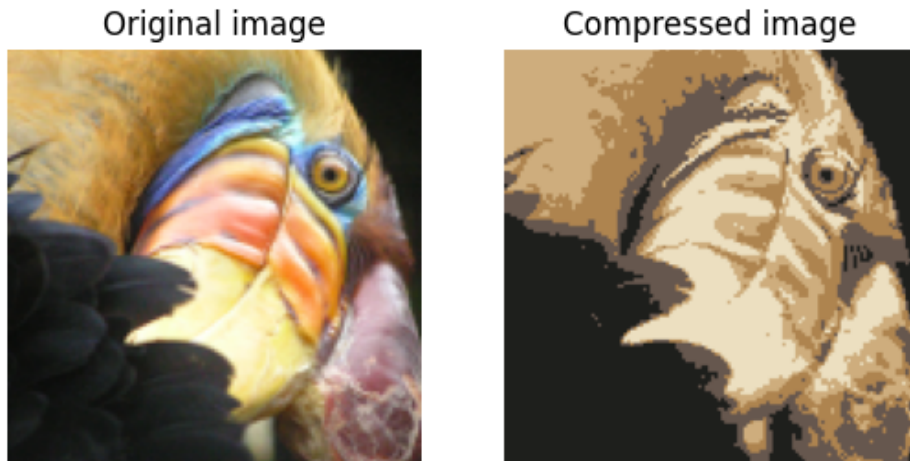


Figure 3: Comparison of the original 24-bit bird image (left) and the K-means compressed version using $K = 5$ colors (right). Reducing the color palette to 5 centroids (5 colors) achieves a large reduction in size, but at the cost of only keeping the most prominent visual structure.

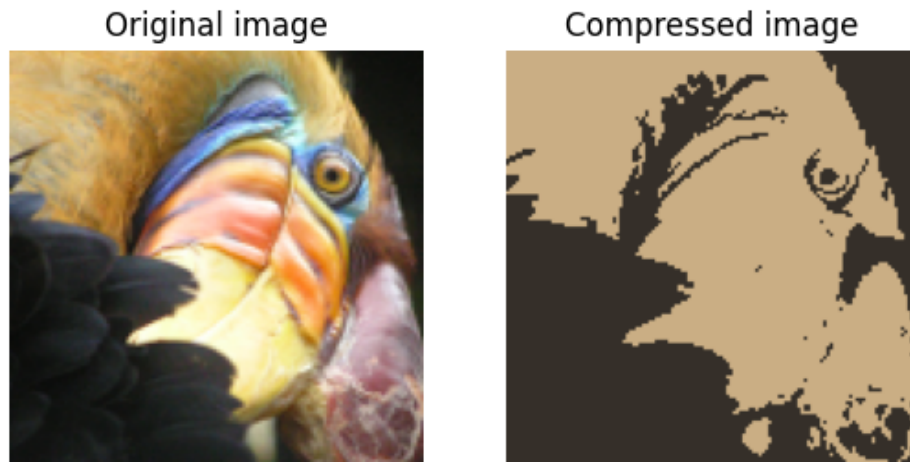


Figure 4: Comparison of the original 24-bit bird image (left) and the K-means compressed version using $K = 2$ colors (right). Reducing the color palette to 2 centroids (2 colors) achieves a massive reduction in size, but at the cost of making the the reconstruction mostly unintelligible.

Conclusion

This lab provided a good way to see K-means clustering in action. In Part A, building the algorithm from scratch made us have to really understand the math/logic behind calculating distances and updating centroids. In Part B, applying it to an image gave a nice way to visualize how clustering works on a larger scale. Treating pixels as data points in a 3D color space and finding the best 16 cluster centers allowed us to compress an image by a factor of 6. Overall, this lab showed that unsupervised clustering is not just for plotting data graphs, but is actually a useful tool for tasks such as dimensionality reduction and data compression.